claude-windows / 56b7f849-5d56-4770-93e0-b3b4da943700

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\56b7f849-5d56-4770-93e0-b3b4da943700\subagents\workflows\wf_d88615a4-f56\agent-aac4a0e5db2cfe047.jsonl`
- SHA-256: `b6fac42b83f5e8194ae69eebf8c2cc757b43c9b6e9ccb4aeec56595cd4bbf4c9`
- Source modified: `2026-06-07T09:11:24+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_d88615a4-f56`
- session_id: `56b7f849-5d56-4770-93e0-b3b4da943700`

Transcript

1. user (2026-06-07T09:10:30.782Z)

Read these files in C:/Users/avidu/Projects/badminton-highlight-indexer and produce a DENSE, anchor-rich CODE_MAP chunk for the "Validators" subsystem.
Use the format: bullet per file/symbol with anchors written as @path::symbol where @path is the repo-relative path EXACTLY as listed (the current layered layout). Capture: key classes/functions/consts; data contracts (mark with §); registries / plugin extension points (mark with ?); and footguns/gotchas (mark with ?). Be precise ? only list symbols that actually exist in the files. If a path is a directory (e.g. tests/), enumerate its files and map each test to the subsystem it covers.
Files/paths: backend/pipeline/validators/__init__.py, backend/pipeline/validators/base.py, backend/pipeline/validators/format.py, backend/pipeline/validators/resolution_fps.py, backend/pipeline/validators/pts_continuity.py, backend/pipeline/validators/scene_cut.py, backend/pipeline/validators/blur.py, backend/pipeline/validators/relevance.py

2. assistant (2026-06-07T09:10:33.322Z)

I'll read all the validator files in parallel to produce the CODE_MAP chunk.

3. user (2026-06-07T09:10:33.778Z)

```
1       from backend.pipeline.validators.base import registry, VideoValidator, ValidationResult
2       from backend.pipeline.validators.format import FormatValidator
3       from backend.pipeline.validators.resolution_fps import ResolutionFPSValidator
4       from backend.pipeline.validators.pts_continuity import PTSContinuityValidator
5       from backend.pipeline.validators.scene_cut import SceneCutDensityValidator
6       from backend.pipeline.validators.blur import BlurValidator
7       from backend.pipeline.validators.relevance import RelevanceValidator
8
9       # Register all default validators in sequence (dependencies first)
10      registry.register(FormatValidator())
11      registry.register(ResolutionFPSValidator())
12      registry.register(PTSContinuityValidator())
13      registry.register(SceneCutDensityValidator())
14      registry.register(BlurValidator())
15      registry.register(RelevanceValidator())
16
17      __all__ = ["registry", "VideoValidator", "ValidationResult"]
18
```

4. user (2026-06-07T09:10:34.747Z)

```
1       from abc import ABC, abstractmethod
2       from pydantic import BaseModel
3       from typing import List, Dict, Any, Optional
4
```

```
5       class ValidationResult(BaseModel):
6           validator_name: str
7           passed: bool
8           message: str
9           details: Dict[str, Any] = {}
10
11      class VideoValidator(ABC):
12          @property
13          @abstractmethod
14          def name(self) -> str:
15              """Unique identifier for this validator."""
16              pass
17
18          @property
19          @abstractmethod
20          def description(self) -> str:
21              """Human-readable explanation of what this validator checks."""
22              pass
23
24          @abstractmethod
25          def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
26              """
27              Executes validation.
28              - video_path: Local path to the MP4 file.
29              - config: Dict parsed from config.json.
30              - context: Dynamic context dict shared across validators (stores shared values
like ffprobe metadata).
31              """
32              pass
33
34      class ValidationRegistry:
35          def __init__(self):
36              self._validators: List[VideoValidator] = []
37
38          def register(self, validator: VideoValidator):
39              """Registers a new validator."""
40              self._validators.append(validator)
41
42          def run_all_with_context(self, video_path: str, config: Dict[str, Any]):
43              """Runs all validators and returns ``(results, context)``.
44
45              The per-run ``context`` accumulates shared values (notably
46              ``context['ffprobe_meta']`` produced by FormatValidator). ``run_all``
47              discards it; callers that want media metadata for an observability report
48              use this variant instead of re-probing.
49              """
50              results = []
51              context = {} # Shared context dictionary (e.g. for caching ffprobe outputs)
52              for val in self._validators:
53                  try:
54                      res = val.validate(video_path, config, context)
55                      results.append(res)
56                  except Exception as e:
57                      results.append(ValidationResult(
58                          validator_name=val.name,
59                          passed=False,
60                          message=f"Validator threw an unexpected error: {str(e)}"
61                      ))
62              return results, context
63
64          def run_all(self, video_path: str, config: Dict[str, Any]) ->
List[ValidationResult]:
65              """Runs all registered validators in sequence, accumulating results."""
66              results, _ = self.run_all_with_context(video_path, config)
```

```
            return results

    # Shared global registry
    registry = ValidationRegistry()

```

5. user (2026-06-07T09:10:35.216Z)

```
    import subprocess
    import json
    import os
    from backend.pipeline.validators.base import VideoValidator, ValidationResult
    from typing import Dict, Any

    class FormatValidator(VideoValidator):
        @property
        def name(self) -> str:
            return "format_check"

        @property
        def description(self) -> str:
            return "Verifies the video is a valid, uncorrupted MP4 container and extracts
metadata."

        def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
            if not os.path.exists(video_path):
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message=f"File not found: {video_path}"
                )

            cmd = [
                "ffprobe",
                "-v", "error",
                "-select_streams", "v:0",
                "-show_format",
                "-show_streams",
                "-of", "json",
                video_path
            ]

            try:
                result = subprocess.run(cmd, capture_output=True, text=True, check=True)
                info = json.loads(result.stdout)

                if not info.get("streams") or not info.get("format"):
                    return ValidationResult(
                        validator_name=self.name,
                        passed=False,
                        message="Failed to retrieve valid video streams from the file."
                    )

                # Store the metadata in context for subsequent validators to utilize
                context["ffprobe_meta"] = info

                stream = info["streams"][0]
                fmt = info["format"]

                details = {
                    "format_name": fmt.get("format_name"),
                    "duration": float(fmt.get("duration", 0)),
                    "size_bytes": int(fmt.get("size", 0)),
                    "codec_name": stream.get("codec_name"),
```

```
                }

                # Check if container name includes mp4
                if "mp4" not in fmt.get("format_name", "").lower():
                    return ValidationResult(
                        validator_name=self.name,
                        passed=False,
                        message=f"Invalid format: Found '{fmt.get('format_name')}', but MP4
is required.",
                        details=details
                    )

                return ValidationResult(
                    validator_name=self.name,
                    passed=True,
                    message="Video format is valid MP4 container.",
                    details=details
                )

            except subprocess.CalledProcessError as e:
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message="Video stream is corrupted or unreadable by ffprobe.",
                    details={"error": e.stderr}
                )
            except Exception as e:
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message=f"Unexpected error running ffprobe: {str(e)}"
                )

```

6. user (2026-06-07T09:10:36.140Z)

```
from backend.pipeline.validators.base import VideoValidator, ValidationResult
from typing import Dict, Any

class ResolutionFPSValidator(VideoValidator):
    @property
    def name(self) -> str:
        return "resolution_fps_check"

    @property
    def description(self) -> str:
        return "Checks if the video meets minimum resolution (e.g. 720p) and frame rate
(e.g. 30fps) requirements."

    def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
        meta = context.get("ffprobe_meta")
        if not meta or not meta.get("streams"):
            return ValidationResult(
                validator_name=self.name,
                passed=False,
                message="Video metadata is missing. Run format_check validator first."
            )

        stream = meta["streams"][0]
        width = int(stream.get("width", 0))
        height = int(stream.get("height", 0))

        # Parse frame rate (e.g. "30/1" or "30000/1001")
        fps_str = stream.get("r_frame_rate", "0/1")
```

```
            try:
                num, den = map(int, fps_str.split("/"))
                fps = num / den if den != 0 else 0
            except ValueError:
                fps = 0.0

            details = {
                "width": width,
                "height": height,
                "fps": round(fps, 2)
            }

            # Check bounds from config
            val_cfg = config.get("validation", {})
            min_w = val_cfg.get("min_resolution_width", 1280)
            min_h = val_cfg.get("min_resolution_height", 720)
            min_fps = val_cfg.get("min_fps", 29.9)

            if width < min_w or height < min_h:
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message=f"Low resolution: Found {width}x{height}, but minimum is
{min_w}x{min_h}.",
                    details=details
                )

            if fps < min_fps:
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message=f"Low frame rate: Found {round(fps, 2)} FPS, but minimum is
{min_fps} FPS.",
                    details=details
                )

            return ValidationResult(
                validator_name=self.name,
                passed=True,
                message="Video resolution and frame rate meet minimum standards.",
                details=details
            )

```

7. user (2026-06-07T09:10:36.632Z)

```
      import subprocess
      from backend.pipeline.validators.base import VideoValidator, ValidationResult
      from typing import Dict, Any

      class PTSContinuityValidator(VideoValidator):
          @property
          def name(self) -> str:
              return "pts_continuity_check"

          @property
          def description(self) -> str:
              return "Analyzes Presentation Time Stamps (PTS) of keyframes to check for
timeline gaps, jumps, or editing splices."

          def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
              # Command to extract packet PTS and flags (extremely fast, works on H.264 &
H.265)
              cmd = [
```

```
                "ffprobe",
                "-v", "error",
                "-select_streams", "v:0",
                "-show_entries", "packet=pts_time,flags",
                "-of", "csv=p=0",
                video_path
            ]

            try:
                result = subprocess.run(cmd, capture_output=True, text=True, check=True)
                output = result.stdout.strip()

                if not output:
                    return ValidationResult(
                        validator_name=self.name,
                        passed=False,
                        message="No packet timestamps could be extracted from the video
stream."
                    )

                timestamps = []
                for line in output.split("\n"):
                    line = line.strip()
                    if line:
                        parts = line.split(",")
                        if len(parts) == 2:
                            pts_str, flags = parts
                            if "K" in flags: # Check if keyframe
                                try:
                                    timestamps.append(float(pts_str))
                                except ValueError:
                                    continue

                if len(timestamps) < 2:
                    return ValidationResult(
                        validator_name=self.name,
                        passed=True,
                        message="Video is too short to perform a PTS continuity check.",
                        details={"keyframes_count": len(timestamps)}
                    )

                # Analyze gaps
                max_gap = 0.0
                backward_jumps = 0
                gaps = []

                for i in range(1, len(timestamps)):
                    gap = timestamps[i] - timestamps[i-1]
                    gaps.append(gap)
                    if gap < 0:
                        backward_jumps += 1
                    if gap > max_gap:
                        max_gap = gap

                # An continuous GoPro recording usually has keyframe intervals (GOP size) of
1.0 to 4.0 seconds.
                # If there's a keyframe gap larger than 8.0 seconds or a negative gap
(backward jump), it's highly likely spliced.
                details = {
                    "keyframes_count": len(timestamps),
                    "max_keyframe_gap_seconds": round(max_gap, 2),
                    "backward_jumps_detected": backward_jumps,
                    "first_keyframe": timestamps[0],
                    "last_keyframe": timestamps[-1]
                }
```

```
79
80                  if backward_jumps > 0:
81                      return ValidationResult(
82                          validator_name=self.name,
83                          passed=False,
84                          message="Video integrity compromise: Backward timeline jumps
(splicing) detected.",
85                          details=details
86                      )
87
88                  # Threshold for sudden gap (e.g. 10.0 seconds)
89                  if max_gap > 10.0:
90                      return ValidationResult(
91                          validator_name=self.name,
92                          passed=False,
93                          message=f"Video integrity compromise: Detected a timestamp gap of
{round(max_gap, 2)}s, indicating edited or missing content.",
94                          details=details
95                      )
96
97                  return ValidationResult(
98                      validator_name=self.name,
99                      passed=True,
100                     message="PTS timeline is continuous with no abnormal gaps or backward
jumps.",
101                     details=details
102                 )
103
104             except subprocess.CalledProcessError as e:
105                 return ValidationResult(
106                     validator_name=self.name,
107                     passed=False,
108                     message="Failed to execute ffprobe PTS analysis.",
109                     details={"error": e.stderr}
110                 )
111             except Exception as e:
112                 return ValidationResult(
113                     validator_name=self.name,
114                     passed=False,
115                     message=f"Unexpected error in PTS validation: {str(e)}"
116                 )
117
```

8. user (2026-06-07T09:10:37.105Z)

```
1       import cv2
2       import numpy as np
3       from backend.pipeline.validators.base import VideoValidator, ValidationResult
4       from typing import Dict, Any
5
6       class SceneCutDensityValidator(VideoValidator):
7           @property
8           def name(self) -> str:
9               return "scene_cut_check"
10
11          @property
12          def description(self) -> str:
13              return "Analyzes visual frames to detect abrupt scene cuts, which indicate an
edited or compiled video."
14
15          def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
16              cap = cv2.VideoCapture(video_path)
17              if not cap.isOpened():
18                  return ValidationResult(
```

```
                    validator_name=self.name,
                    passed=False,
                    message="Failed to open video file using OpenCV."
                )

            fps = cap.get(cv2.CAP_PROP_FPS)
            total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
            duration = total_frames / fps if fps > 0 else 0

            if duration <= 0:
                cap.release()
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message="Invalid video duration."
                )

            # Sample at 2 frames per second to keep it fast
            sample_interval = max(1, int(fps / 2))

            cuts_detected = 0
            prev_hist = None

            frame_idx = 0
            sampled_count = 0

            # Adjust histogram comparison thresholds
            # Using correlation: 1.0 is perfect match, < 0.6 indicates a sudden massive
scene change
            cut_threshold = 0.6

            while True:
                cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
                ret, frame = cap.read()
                if not ret:
                    break

                # Calculate color histogram
                hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
                hist = cv2.calcHist([hsv], [0, 1], None, [50, 60], [0, 180, 0, 256])
                cv2.normalize(hist, hist, alpha=0, beta=1, norm_type=cv2.NORM_MINMAX)

                if prev_hist is not None:
                    # Compare correlation
                    score = cv2.compareHist(prev_hist, hist, cv2.HISTCMP_CORREL)
                    if score < cut_threshold:
                        cuts_detected += 1

                prev_hist = hist
                sampled_count += 1
                frame_idx += sample_interval

                # Cap check at 100 frames max (e.g. first 50 seconds or evenly spread) to
keep validation instant
                if frame_idx >= total_frames or sampled_count > 100:
                    break

            cap.release()

            # Compute cuts per minute
            sampled_duration = (sampled_count * sample_interval) / fps if fps > 0 else 1
            cuts_per_minute = (cuts_detected / sampled_duration) * 60 if sampled_duration >
0 else 0

            details = {
```

```
                "cuts_detected": cuts_detected,
                "sampled_frames": sampled_count,
                "cuts_per_minute": round(cuts_per_minute, 3)
            }

            val_cfg = config.get("validation", {})
            max_cuts_rate = val_cfg.get("max_scene_cuts_per_minute", 0.5)

            if cuts_per_minute > max_cuts_rate:
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message=f"Video splicing compromise: Detected {cuts_detected} cuts
({round(cuts_per_minute, 2)} cuts/min), which exceeds the continuous raw file limit of
{max_cuts_rate} cuts/min.",
                    details=details
                )

            return ValidationResult(
                validator_name=self.name,
                passed=True,
                message="Scene continuity check passed. No abnormal visual cuts detected.",
                details=details
            )

```

9. user (2026-06-07T09:10:38.047Z)

```
      import cv2
      from backend.pipeline.validators.base import VideoValidator, ValidationResult
      from typing import Dict, Any

      class BlurValidator(VideoValidator):
          @property
          def name(self) -> str:
              return "blur_check"

          @property
          def description(self) -> str:
              return "Samples frames across the video and checks if the footage is excessively
blurry or out of focus."

          def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
              cap = cv2.VideoCapture(video_path)
              if not cap.isOpened():
                  return ValidationResult(
                      validator_name=self.name,
                      passed=False,
                      message="Failed to open video file for blur analysis."
                  )

              total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
              if total_frames <= 0:
                  cap.release()
                  return ValidationResult(
                      validator_name=self.name,
                      passed=False,
                      message="Invalid frame count."
                  )

              # Sample 15 frames evenly distributed across the video
              num_samples = 15
              step = max(1, total_frames // num_samples)

```

```
            variances = []

            for i in range(num_samples):
                frame_idx = i * step
                cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
                ret, frame = cap.read()
                if not ret:
                    continue

                gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
                # Calculate the Laplacian variance (standard focus metric)
                laplacian_var = cv2.Laplacian(gray, cv2.CV_64F).var()
                variances.append(laplacian_var)

            cap.release()

            if not variances:
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message="Could not read frames to perform blur verification."
                )

            avg_variance = sum(variances) / len(variances)

            details = {
                "avg_laplacian_variance": round(avg_variance, 2),
                "min_laplacian_variance": round(min(variances), 2),
                "max_laplacian_variance": round(max(variances), 2),
                "sampled_frames_count": len(variances)
            }

            val_cfg = config.get("validation", {})
            min_blur_threshold = val_cfg.get("min_blur_threshold", 100.0)

            if avg_variance < min_blur_threshold:
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message=f"Video quality rejected: Focus check failed. Average sharpness
score of {round(avg_variance, 1)} is below the minimum required threshold of
{min_blur_threshold}.",
                    details=details
                )

            return ValidationResult(
                validator_name=self.name,
                passed=True,
                message="Video sharpness meets focus requirements.",
                details=details
            )

```

10. user (2026-06-07T09:10:38.194Z)

```
    import cv2
    import numpy as np
    from backend.pipeline.validators.base import VideoValidator, ValidationResult
    from typing import Dict, Any

    class RelevanceValidator(VideoValidator):
        @property
        def name(self) -> str:
            return "relevance_check"

```

```
        @property
        def description(self) -> str:
            return "Checks if the video content is relevant to badminton by scanning for court color palettes and geometric features."

        def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any]) -> ValidationResult:
            cap = cv2.VideoCapture(video_path)
            if not cap.isOpened():
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message="Failed to open video file for relevance validation."
                )

            total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
            if total_frames <= 0:
                cap.release()
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message="Invalid frame count."
                )

            # Sample 10 frames across the video
            num_samples = 10
            step = max(1, total_frames // num_samples)

            green_ratios = []

            # Get sport profile config
            sport_name = config.get("sport", "badminton")
            from backend.sports import sport_registry
            try:
                sport_profile = sport_registry.get(sport_name)
                sport_rel_cfg = sport_profile.get_relevance_config()
            except KeyError:
                sport_rel_cfg = {}

            val_cfg = config.get("validation", {})
            rel_cfg = val_cfg.get("relevance", {})

            # Read bounds from sport profile with fallback to user config/defaults
            hsv_lower = np.array(rel_cfg.get("court_green_lower", sport_rel_cfg.get("court_green_lower", [35, 40, 40])))
            hsv_upper = np.array(rel_cfg.get("court_green_upper", sport_rel_cfg.get("court_green_upper", [85, 255, 255])))
            min_court_ratio = rel_cfg.get("court_pixels_min_ratio", sport_rel_cfg.get("court_pixels_min_ratio", 0.05))

            for i in range(num_samples):
                frame_idx = i * step
                cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
                ret, frame = cap.read()
                if not ret:
                    continue

                # Downscale frame for speed
                h, w = frame.shape[:2]
                small_frame = cv2.resize(frame, (320, 240))

                # Convert to HSV color space
                hsv = cv2.cvtColor(small_frame, cv2.COLOR_BGR2HSV)

                # Mask court green pixels
```

```
71                  mask = cv2.inRange(hsv, hsv_lower, hsv_upper)
72                  green_pixels = cv2.countNonZero(mask)
73                  total_pixels = mask.shape[0] * mask.shape[1]
74
75                  ratio = green_pixels / total_pixels
76                  green_ratios.append(ratio)
77
78              cap.release()
79
80              if not green_ratios:
81                  return ValidationResult(
82                      validator_name=self.name,
83                      passed=False,
84                      message="Failed to extract frames for relevance checking."
85                  )
86
87              avg_green_ratio = sum(green_ratios) / len(green_ratios)
88
89              details = {
90                  "avg_green_court_ratio": round(avg_green_ratio, 4),
91                  "max_green_court_ratio": round(max(green_ratios), 4),
92                  "min_green_court_ratio": round(min(green_ratios), 4),
93                  "green_threshold": min_court_ratio
94              }
95
96              # If a certain amount of court color is visible, consider it relevant
97              if avg_green_ratio < min_court_ratio:
98                  return ValidationResult(
99                      validator_name=self.name,
100                     passed=False,
101                     message=f"Video rejected: Relevance check failed. The scene does not
contain a recognizable {sport_name} court (court area found: {round(avg_green_ratio * 100, 2)}%
vs min required {round(min_court_ratio * 100, 2)}%).",
102                     details=details
103                 )
104
105             return ValidationResult(
106                 validator_name=self.name,
107                 passed=True,
108                 message=f"Video identified as relevant to {sport_name} ({sport_name} court
structures found).",
109                 details=details
110             )
111
```

11. assistant (2026-06-07T09:11:23.981Z)

Validators subsystem

Layered validation pipeline (`backend/pipeline/validators/`): each validator is a plugin implementing the `VideoValidator` ABC, registered into a global ordered `registry`, and run in sequence against an MP4 with a shared mutable `context` dict (used to pass ffprobe metadata downstream). All validators return a uniform `ValidationResult`.

base ? contracts + registry

- §`ValidationResult` @backend/pipeline/validators/base.py::ValidationResult ? pydantic `BaseModel`; the uniform output contract. Fields: `validator_name: str`, `passed: bool`, `message: str`, `details: Dict[str, Any] = {}`. ? `details` default `{}` is a shared mutable class-level default (Pydantic copies per-instance so safe here, but note it's the contract every validator populates).
- ? `VideoValidator` @backend/pipeline/validators/base.py::VideoValidator ? ABC plugin base. Abstract `@property name -> str` (unique id), abstract `@property description -> str`, abstract `validate(self, video_path: str, config: Dict, context: Dict) -> ValidationResult`. §`validate` signature is THE plugin contract: `config` is parsed `config.json`; `context` is the shared per-

run dict.
- ? `ValidationRegistry` @backend/pipeline/validators/base.py::ValidationRegistry ? ordered registry (`self._validators: List[VideoValidator]`, a list not a map ? registration ORDER is execution order).
  - `register(validator)` @backend/pipeline/validators/base.py::ValidationRegistry.register ? appends; ? no dedup / no name-collision check, duplicates run twice.
  - §`run_all_with_context(video_path, config) -> (results, context)` @backend/pipeline/validators/base.py::ValidationRegistry.run_all_with_context ? runs every validator in sequence, accumulating `context`; wraps each `validate` in try/except so one validator's exception becomes a failed `ValidationResult` instead of aborting the run. ? the exception-path `ValidationResult` omits `message`?? no ? sets `message` but NOT `details`; uses `val.name`. Callers wanting media metadata read the returned `context['ffprobe_meta']` to avoid re-probing.
  - `run_all(video_path, config) -> List[ValidationResult]` @backend/pipeline/validators/base.py::ValidationRegistry.run_all ? thin wrapper that discards `context`.
  - ? `registry` @backend/pipeline/validators/base.py::registry ? module-level singleton `ValidationRegistry()`; the global extension point.

- ? @backend/pipeline/validators/__init__.py ? import-time side-effect wiring: instantiates and `registry.register(...)`s the 6 default validators in dependency order: `FormatValidator` ? `ResolutionFPSValidator` ? `PTSContinuityValidator` ? `SceneCutDensityValidator` ? `BlurValidator` ? `RelevanceValidator`. Re-exports `registry`, `VideoValidator`, `ValidationResult` via `__all__`. ? ORDER MATTERS ? `FormatValidator` must run first because it populates `context['ffprobe_meta']` that `ResolutionFPSValidator` depends on. ? importing this module mutates the global `registry` as a side effect; importing twice would re-register (idempotency relies on Python module caching).

Validators (each implements VideoValidator)
- `FormatValidator` @backend/pipeline/validators/format.py::FormatValidator ? name `"format_check"`. Runs `ffprobe -show_format -show_streams -of json` on `v:0`. ? PRODUCER: stores full probe JSON into §`context["ffprobe_meta"]` for downstream validators. Returns `details={format_name, duration(float), size_bytes(int), codec_name}`. Fails on: missing file (`os.path.exists`), empty streams/format, `format_name` not containing `"mp4"` (substring, case-insensitive), `CalledProcessError` (corrupt/unreadable). ? MP4 check is a loose substring match on `format_name` (ffprobe reports e.g. `"mov,mp4,m4a,3gp,3g2,mj2"` so most MOV-family files pass). ? external `ffprobe` binary dependency.
- `ResolutionFPSValidator` @backend/pipeline/validators/resolution_fps.py::ResolutionFPSValidator ? name `"resolution_fps_check"`. CONSUMER: reads `context.get("ffprobe_meta")`; fails fast with "Run format_check validator first" if absent. Parses `width`/`height` and `r_frame_rate` (`"num/den"` rational). §config keys under `config["validation"]`: `min_resolution_width` (default 1280), `min_resolution_height` (default 720), `min_fps` (default 29.9). `details={width,height,fps}`. ? depends on FormatValidator having run first (ordering coupling via context). ? `fps` parse catches only `ValueError` ? 0.0; a non-`"a/b"` string lacking `/` raises elsewhere but split form is assumed.
- `PTSContinuityValidator` @backend/pipeline/validators/pts_continuity.py::PTSContinuityValidator ? name `"pts_continuity_check"`. Independently shells `ffprobe -show_entries packet=pts_time,flags -of csv=p=0` (does NOT use `context`; re-probes). Keeps only keyframe packets (`"K" in flags`), computes `max_keyframe_gap_seconds`, `backward_jumps_detected`. Fails on: any backward jump, or `max_gap > 10.0`s (? hard-coded threshold, NOT config-driven ? comment mentions 8.0s but code uses 10.0). PASSES (not fails) if `< 2` keyframes ("too short"). `details={keyframes_count,max_keyframe_gap_seconds,backward_jumps_detected,first_keyframe,last_keyframe}`. ? keyframe gap analysis assumes raw continuous capture (GOP 1?4s).
- `SceneCutDensityValidator` @backend/pipeline/validators/scene_cut.py::SceneCutDensityValidator ? name `"scene_cut_check"`. OpenCV (`cv2.VideoCapture`). Samples ~2 fps (`sample_interval = max(1, int(fps/2))`), HSV H-S histogram `[50,60]` bins, `cv2.compareHist(..., HISTCMP_CORREL)`; a frame-to-frame correlation `< cut_threshold` (? hard-coded `0.6`, NOT config) counts as a cut. ? HARD CAP: stops after 100 sampled frames (`sampled_count > 100`) ? only first ~50s analyzed on long videos. §config `config["validation"]["max_scene_cuts_per_minute"]` (default 0.5) is the fail threshold on computed `cuts_per_minute`.
`details={cuts_detected,sampled_frames,cuts_per_minute}`. ? `cap.set(CAP_PROP_POS_FRAMES, frame_idx)` per-frame seeking is slow/unreliable on some codecs.
- `BlurValidator` @backend/pipeline/validators/blur.py::BlurValidator ? name `"blur_check"`.

OpenCV; samples 15 evenly-spaced frames (`step = max(1, total_frames//15)`), computes Laplacian variance (`cv2.Laplacian(gray, CV_64F).var()`) as focus metric, averages. §config `config["validation"]["min_blur_threshold"]` (default 100.0); fails if `avg_variance < threshold`. `details={avg_laplacian_variance,min_?,max_?,sampled_frames_count}`. ? Laplacian-variance threshold is scene/resolution dependent ? 100.0 is a magic absolute value not normalized to resolution.
- `RelevanceValidator` @backend/pipeline/validators/relevance.py::RelevanceValidator ? name `"relevance_check"`. OpenCV; samples 10 frames, downscales to 320×240, HSV `cv2.inRange` court-green mask, computes per-frame green-pixel ratio, averages. ? INTEGRATION POINT: imports `from backend.sports import sport_registry` (lazy, inside `validate`), resolves `config.get("sport","badminton")` ? `sport_profile.get_relevance_config()`; on `KeyError` falls back to `{}`. §config precedence (3-tier): `config["validation"]["relevance"][key]` ? `sport_rel_cfg[key]` ? hard-coded default. Keys: `court_green_lower` (default `[35,40,40]`), `court_green_upper` (default `[85,255,255]`), `court_pixels_min_ratio` (default 0.05). `details={avg_green_court_ratio,max_?,min_?,green_threshold}`. ? "relevance" is purely color-heuristic (green pixel ratio) despite description mentioning "geometric features" ? no geometry is actually computed. ? assumes green courts; blue/wood badminton courts would fail. ? cross-subsystem coupling to `backend.sports.sport_registry`.

Cross-cutting notes

- §Shared `context` dict is the only inter-validator channel; only `FormatValidator` writes it (`ffprobe_meta`), only `ResolutionFPSValidator` reads it. PTS/SceneCut/Blur/Relevance each re-open the file independently (ffprobe or OpenCV) ? multiple decode passes per video.
- ? Three external dependencies: `ffprobe` CLI (format, pts_continuity) and `cv2`/`numpy` (scene_cut, blur, relevance). No availability guard ? missing binary surfaces as a per-validator failed result via the registry try/except.
- ? Config schema is implicit/duck-typed: all thresholds live under `config["validation"]` (and `["validation"]["relevance"]`) with inline defaults scattered across files ? no central schema. Two thresholds (PTS 10.0s gap, scene-cut 0.6 correlation) are NOT config-overridable.

Note: I did not find a `tests/` directory among the provided paths (none was listed), so no test-to-subsystem mapping is included. The relevant files are all under `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/validators/`.